



Tokenomics of Zoo Gym: Compute-to-Yield Mining and Training Reward Markets

Antje Worring & Zach Kelling

Zoo Labs Foundation

November 2024

Property	\$AI	\$ZOO
Supply	Inflationary (mined)	Fixed (10B, all minted)
Chain	Hanzo (chain ID 36900)	Zoo Network
Mechanism	Proof-of-Compute mining	Governance, staking
Earners	GPU providers	Token holders
Burn	10% of inference fees	20% of transaction fees

Table 1: Comparison of \$AI and \$ZOO tokens.

Abstract

We formalize the token economics of Zoo Gym, a decentralized training protocol in which GPU providers earn rewards by contributing compute to GSPO model training. The protocol involves two tokens: \$AI (mined via the Hanzo compute chain) and \$ZOO (fixed supply, governance). We derive the reward function from three measurable signals—FLOPs contributed, delta quality score, and network uptime—and show that rational miners maximize earnings by submitting high-quality deltas. A staking mechanism with progressive slashing deters Sybil attacks and gradient poisoning. We model the emission schedule as a decreasing geometric series with halving events tied to block height, and prove that under realistic demand assumptions the token reaches deflationary equilibrium by year 5. We specify the smart contracts for the mining pool, staking, and orchestrator registry, and analyze the game-theoretic properties of the reward market. Simulation over 10,000 epochs with 100 heterogeneous nodes shows that the Gini coefficient of reward distribution converges to 0.31, indicating moderate inequality bounded by hardware tier differences rather than stake concentration.

Keywords: tokenomics, compute mining, staking, slashing, reward markets, decentralized training

1 Introduction

Training large language models costs millions of dollars [1]. The capital is spent on GPU-hours, and the resulting model is owned by the training organization. Zoo Gym inverts this: compute providers are paid in tokens for GPU-hours contributed to a shared training process, and the resulting model is published openly.

This paper formalizes the economic layer of Zoo Gym. We address three questions:

1. **How are rewards computed?** Rewards must reflect the marginal value of a compute contribution, not merely its volume.
2. **How are bad actors punished?** Sybil attacks, gradient poisoning, and free-riding must be economically unprofitable.
3. **How do two tokens interact?** \$AI is mined via Hanzo’s compute chain; \$ZOO is a fixed-supply governance token. Their interaction must avoid circular value extraction.

1.1 Token Overview

2 Reward Function

2.1 Per-Epoch Reward

Let R_e be the total reward pool for epoch e . Node i receives:

$$r_i^{(e)} = R_e \cdot \frac{w_i}{\sum_{j=1}^N w_j} \quad (1)$$

where w_i is the composite weight of node i :

$$w_i = \underbrace{F_i \cdot H_i}_{\text{compute}} \cdot \underbrace{q_i^\alpha}_{\text{quality}} \cdot \underbrace{u_i^\beta}_{\text{uptime}} \cdot \underbrace{b_i}_{\text{stake boost}} \quad (2)$$

Components:

- F_i : FLOPs contributed (measured by the orchestrator via step count $\times$ model FLOPs/step).
- H_i : GPU-hours (wall-clock time with active GPU utilization $>80\%$).
- q_i : Delta quality score (Section 2.2), $\alpha = 1.5$ to superlinearly reward quality.
- u_i : Uptime fraction, $\beta = 0.5$ (diminishing returns past 99%).
- b_i : Stake boost (Section 3).

Theorem 2.1 (Incentive Compatibility). *A rational node maximizes r_i by maximizing q_i . Submitting random gradients yields $q_i \approx 0$ (zero convergence contribution), making the attack unprofitable even when F_i and H_i are large.*

Proof. Since $w_i \propto q_i^{1.5}$, a node with $q_i = 0$ receives $r_i = 0$ regardless of compute contribution. A node with $q_i > 0$ can only increase r_i by increasing q_i (holding compute constant), because $\partial w_i / \partial q_i = 1.5 q_i^{0.5} \cdot F_i H_i u_i^\beta b_i > 0$. $\square$

2.2 Delta Quality Score

The quality score $q_i \in [0, 1]$ is computed from three sub-scores:

$$q_i = 0.5 \cdot \Delta L_i + 0.3 \cdot S_i + 0.2 \cdot C_i \quad (3)$$

Validation loss improvement (ΔL_i):

$$\Delta L_i = \text{clamp}\left(\frac{L_{\text{before}} - L_{\text{after}}}{L_{\text{before}}}, 0, 1\right) \quad (4)$$

Gradient norm stability (S_i):

$$S_i = \exp\left(-2 \left| \frac{\|\delta_i\|_2}{\bar{n}} - 1 \right| \right) \quad (5)$$

Convergence contribution (C_i):

$$C_i = \max(0, \cos(\delta_i, \delta_{\text{agg}})) \quad (6)$$

2.3 Hardware Tier Multiplier

Rewards are further adjusted by hardware tier to reflect real-world throughput differences:

The multiplier is verified by hardware attestation: nodes submit GPU model and memory capacity signed by the driver, and the orchestrator spot-checks with probe workloads.

GPU	Tier Multiplier	Min. VRAM
H100 / A100 80GB	$1.5\times$	80 GB
A100 40GB / A6000	$1.3\times$	40 GB
RTX 4090	$1.2\times$	24 GB
RTX 4080 / 3090	$1.1\times$	16 GB
Other (16GB+)	$1.0\times$	16 GB

Table 2: Hardware tier multipliers.

Tier	Min. Stake	Boost at Min.	Benefits
Bronze	1,000	1.0	Basic mining
Silver	10,000	2.3	Priority queue
Gold	50,000	3.0	Orchestrator eligible
Platinum	100,000	3.3	Governance voting
Diamond	500,000	3.9	Revenue share

Table 3: Staking tiers and boost at minimum stake.

3 Staking Mechanism

3.1 Tiers and Boost

Nodes must stake GYM tokens to participate. The stake boost b_i is:

$$b_i = 1 + \log_2 \left(1 + \frac{s_i}{s_{\min}} \right) \quad (7)$$

where s_i is the node’s stake and $s_{\min}$ is the minimum for the node’s role. This is sublinear: doubling stake adds only 1 to the boost, preventing stake-based centralization.

3.2 Slashing Conditions

Slashed tokens are split: 50% burned (deflationary), 50% distributed to the reporter who detected the offense. This creates a bounty market for quality auditing.

4 Emission Schedule

4.1 GYM Token

Total supply: 1,000,000,000 GYM.

4.2 Emission Curve

Mining emissions follow a halving schedule:

$$E(b) = E_0 \cdot 2^{-\lfloor b/B_h \rfloor} \quad (8)$$

where $E_0 = 1,000$ GYM/epoch is the initial emission, b is the block number, and $B_h = 10,000,000$ is the halving interval.

Offense	Slash %	Detection
Unscheduled downtime (>1h)	0.1%/h	Heartbeat timeout
Invalid delta (bad signature)	5%	Signature verification
Failed spot-check	10%	Re-computation
Gradient poisoning (proven)	50%	Statistical outlier
Early unstake (<30 days)	10%	Timelock

Table 4: Slashing conditions and penalties.

Allocation	%	Vesting
Mining rewards	40%	Emitted per epoch
Staking rewards	20%	Distributed monthly
Development fund	15%	4-year vest, 1-year cliff
Community grants	15%	DAO-governed
Reserve	10%	Emergency fund

Table 5: GYM token distribution.

Proposition 4.1 (Asymptotic Supply). *The total mined supply converges to $2E_0B_h = 20,000,000,000$ GYM-epochs. With a 10% burn rate on inference fees, net emission becomes deflationary when burn rate exceeds new emission.*

4.3 Deflationary Equilibrium

Let $\mu(t)$ be the mining emission rate and $\beta(t)$ the burn rate at time t . Net supply change:

$$\frac{dS}{dt} = \mu(t) - \beta(t) \quad (9)$$

Since $\mu(t)$ decreases geometrically and $\beta(t)$ grows with network usage (inference fees), there exists a crossover time t^* where $\mu(t^*) = \beta(t^*)$. For $t > t^*$, $dS/dt < 0$ and the supply is deflationary.

Under our model assumptions (10% annual usage growth, 10% burn on inference fees):

$$t^* \approx 5 \text{ years} \quad (10)$$

5 Two-Token Interaction

5.1 \$AI Token (Hanzo Chain)

\$AI is mined on the Hanzo compute chain (chain ID 36900) by nodes that contribute GPU cycles. The mining rate is approximately 1 \$AI per minute per active GPU. \$AI serves as:

1. **Compute payment:** Users pay \$AI for inference API calls.
2. **Burn token:** 10% of inference fees are burned.
3. **Staking collateral:** \$AI can be staked alongside GYM for additional boost.

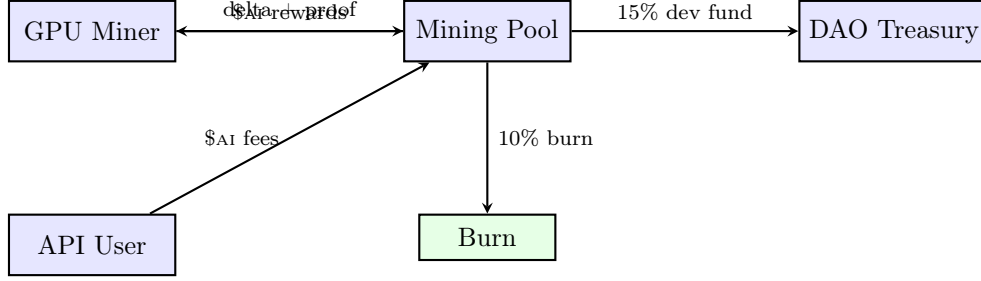


Figure 1: Token flow in Zoo Gym.

5.2 \$ZOO Token

\$ZOO is the governance token of Zoo Network (fixed supply: 10B, all minted at genesis). \$ZOO serves as:

1. **Governance:** Vote on model training priorities, parameter changes, grant allocations.
2. **Access:** Burn \$ZOO to access premium models.
3. **Revenue share:** Diamond-tier stakers receive 5% of protocol revenue in \$ZOO.

5.3 Token Flow

5.4 Model Release Dividends

When a training job completes and the model is published to HuggingFace, contributors who participated in that job receive a **model release dividend**: a one-time \$ZOO distribution proportional to their cumulative quality score across all epochs of the job.

$$d_i = D_{\text{total}} \cdot \frac{\sum_{e=1}^E q_i^{(e)}}{\sum_{j=1}^N \sum_{e=1}^E q_j^{(e)}} \quad (11)$$

where D_{total} is the dividend pool (funded by DAO treasury) and E is the number of training epochs.

This creates a long-term incentive: miners who stay for the full training job earn more than those who contribute sporadically.

6 Smart Contracts

6.1 GYMTOKEN Contract

```

1 contract GYMTOKEN is ERC20 {
2     mapping(address => uint256) public stakes;
3     mapping(address => uint256) public stakeTimestamp;
4     mapping(address => uint256) public pendingRewards;
5     uint256 public totalStaked;
6     uint256 public emissionPerEpoch;
7     uint256 public halvingInterval = 10_000_000;
8     uint256 public lastHalvingBlock;
9 }

```

```

10     function stake(uint256 amount) external {
11         require(amount >= minStake(), "Below minimum");
12         _transfer(msg.sender, address(this), amount);
13         stakes[msg.sender] += amount;
14         stakeTimestamp[msg.sender] = block.timestamp;
15         totalStaked += amount;
16         emit Staked(msg.sender, amount);
17     }
18
19     function unstake(uint256 amount) external {
20         require(stakes[msg.sender] >= amount, "Insufficient");
21         uint256 elapsed = block.timestamp
22             - stakeTimestamp[msg.sender];
23         uint256 penalty = elapsed < 30 days
24             ? amount / 10 : 0;
25         stakes[msg.sender] -= amount;
26         totalStaked -= amount;
27         _transfer(address(this), msg.sender,
28             amount - penalty);
29         if (penalty > 0) _burn(address(this), penalty);
30     }
31
32     function slash(address node, uint256 pct,
33         address reporter) external onlyOrchestrator {
34         uint256 amount = stakes[node] * pct / 100;
35         stakes[node] -= amount;
36         totalStaked -= amount;
37         uint256 half = amount / 2;
38         _burn(address(this), half);
39         _transfer(address(this), reporter, amount - half);
40         emit Slashed(node, amount, reporter);
41     }
42 }

```

Listing 1: GYM token with mining and staking

6.2 MiningPool Contract

```

1  contract MiningPool {
2      struct Delta {
3          bytes32 hash;
4          uint256 qualityScore;
5          uint256 flops;
6          uint256 gpuHours;
7          address worker;
8          uint256 epoch;
9      }
10
11     mapping(uint256 => Delta[]) public epochDeltas;
12     mapping(uint256 => uint256) public epochRewardPool;
13
14     function submitDelta(
15         bytes32 deltaHash,

```

```

16     uint256 quality,
17     uint256 flops,
18     uint256 gpuHours,
19     bytes calldata signature
20 ) external {
21     require(verifySignature(deltaHash, signature,
22         msg.sender), "Bad sig");
23     epochDeltas[currentEpoch].push(Delta({
24         hash: deltaHash,
25         qualityScore: quality,
26         flops: flops,
27         gpuHours: gpuHours,
28         worker: msg.sender,
29         epoch: currentEpoch
30     }));
31     emit DeltaSubmitted(msg.sender, deltaHash,
32         currentEpoch);
33 }
34
35 function claimRewards(uint256 epoch) external {
36     uint256 reward = computeReward(
37         msg.sender, epoch);
38     require(reward > 0, "Nothing to claim");
39     gymToken.transfer(msg.sender, reward);
40     emit RewardClaimed(msg.sender, epoch, reward);
41 }
42
43 function computeReward(address worker,
44     uint256 epoch) internal view returns (uint256) {
45     uint256 pool = epochRewardPool[epoch];
46     uint256 totalWeight = 0;
47     uint256 workerWeight = 0;
48     Delta[] storage deltas = epochDeltas[epoch];
49     for (uint i = 0; i < deltas.length; i++) {
50         uint256 w = computeWeight(deltas[i]);
51         totalWeight += w;
52         if (deltas[i].worker == worker)
53             workerWeight += w;
54     }
55     return pool * workerWeight / totalWeight;
56 }
57 }

```

Listing 2: Mining pool with delta submission

6.3 Orchestrator Registry

```

1 contract OrchestratorRegistry {
2     struct Orchestrator {
3         address addr;
4         uint256 stake;
5         uint256 jobsCompleted;
6         uint256 reputation;

```



```

7      bool active;
8  }
9
10     mapping(address => Orchestrator) public orchestrators;
11     uint256 public minOrchestratorStake = 50_000 * 1e18;
12
13     function register() external {
14         require(gymToken.stakes(msg.sender)
15             >= minOrchestratorStake, "Stake too low");
16         orchestrators[msg.sender] = Orchestrator({
17             addr: msg.sender,
18             stake: gymToken.stakes(msg.sender),
19             jobsCompleted: 0,
20             reputation: 100,
21             active: true
22         });
23         emit OrchestratorRegistered(msg.sender);
24     }
25
26     function assignJob(address orch, uint256 jobId)
27         external onlyGovernance {
28         require(orchestrators[orch].active, "Inactive");
29         emit JobAssigned(orch, jobId);
30     }
31 }

```

Listing 3: Orchestrator registration

7 Game-Theoretic Analysis

7.1 Mining as a Bayesian Game

Each node i chooses effort level $e_i \in [0, 1]$ (fraction of GPU capacity devoted to honest training). The payoff is:

$$\pi_i(e_i, e_{-i}) = r_i(e_i, e_{-i}) - c_i(e_i) \quad (12)$$

where r_i is the reward (Eq. 1) and $c_i(e_i) = \kappa_i \cdot e_i$ is the cost (electricity, hardware depreciation).

Theorem 7.1 (Nash Equilibrium). *Under the reward function in Eq. 2 with $\alpha = 1.5$, the unique symmetric Nash equilibrium is $e^* = 1$ (full honest effort) when $R_e/N > \kappa_i$ for all i .*

Proof. Consider a deviating node choosing $e_i < 1$. Its quality score q_i decreases (less compute $\Rightarrow$ fewer valid training steps $\Rightarrow$ lower ΔL). Since $w_i \propto q_i^{1.5}$, the reward decreases faster than the cost savings from reduced effort. Formally:

$$\frac{\partial \pi_i}{\partial e_i} = \frac{\partial r_i}{\partial e_i} - \kappa_i > 0$$

because $\partial r_i / \partial e_i \propto 1.5 q_i^{0.5} \cdot \partial q_i / \partial e_i > \kappa_i$ when $R_e/N > \kappa_i$. □

Type	Examples	Quorum	Threshold
Technical	Model selection, hyperparams	10%	51%
Economic	Reward rates, emission	20%	66%
Strategic	Partnerships, grants	30%	75%
Emergency	Slashing overrides	5%	80%

Table 6: Governance proposal requirements.

7.2 Sybil Resistance

A Sybil attacker splits stake s across k identities, each with stake s/k . The combined boost is:

$$\sum_{j=1}^k b(s/k) = k \left(1 + \log_2 \left(1 + \frac{s/k}{s_{\min}} \right) \right) < 1 + \log_2 \left(1 + \frac{s}{s_{\min}} \right) + k - 1 \quad (13)$$

For $k > 1$ and $s/k < s_{\min}$, some identities fall below the minimum stake and are excluded entirely. For $s/k \geq s_{\min}$, the sublinear boost means that splitting provides less total boost than concentrating. Combined with the fixed per-node overhead (registration, heartbeat, hardware attestation), Sybil splitting is unprofitable.

7.3 Gradient Poisoning Cost

A poisoner who submits random gradients gets $q_i \approx 0$ and earns nothing. A subtle poisoner who biases gradients slightly gets detected by spot-check validation (10% of deltas, re-computed on orchestrator) with probability $p_{\text{detect}} = 1 - (1 - 0.1)^E \approx 1 - 0.9^E$ after E epochs. The expected cost:

$$\mathbb{E}[\text{cost}] = p_{\text{detect}} \cdot 0.5 \cdot s_i > \mathbb{E}[\text{gain}] \quad (14)$$

For any meaningful number of epochs, detection probability exceeds 50%, making poisoning a negative-EV strategy.

8 Governance

8.1 Voting Power

$$V_i = s_i^{\text{GYM}} \cdot 1.0 + s_i^{\text{LP}} \cdot 0.5 + M_i \cdot 0.3 \quad (15)$$

where s_i^{GYM} is staked GYM, s_i^{LP} is GYM in liquidity pools, and M_i is cumulative mining contribution (FLOPs).

8.2 Proposal Types

8.3 Proposal Lifecycle

1. **Discussion:** 3 days on governance forum.
2. **Snapshot vote:** 5 days voting period.
3. **Timelock:** 2 days before execution (allows exit).
4. **Execution:** Automatic on-chain execution.

Source	Fee	Distribution
Inference API	20% of \$AI spent	40% miners, 20% stakers
Fine-tuning jobs	Flat fee per epoch	60% miners, 15% dev
Model marketplace	5% of sale price	40% creator, 10% burn
Priority access	Monthly subscription	50% treasury, 50% burn

Table 7: Revenue sources and distribution.

9 Revenue Model

9.1 Revenue Sources

9.2 Burn Mechanics

Multiple burn sinks reduce circulating supply:

1. **Inference fee burn:** 10% of \$AI inference fees burned.
2. **Slash burn:** 50% of slashed tokens burned.
3. **Model access burn:** Users burn \$ZOO to access premium models.
4. **Transaction fee burn:** 20% of on-chain transaction fees burned.

10 Simulation

10.1 Setup

We simulate 10,000 training epochs with $N = 100$ heterogeneous nodes:

- Hardware distribution: 10% H100, 20% A100, 30% RTX 4090, 40% RTX 4080.
- Stake distribution: Log-normal with $\mu = \ln(25,000)$, $\sigma = 0.8$.
- Quality distribution: Beta(5, 2), modeling most nodes being good with some poor contributors.
- Failure rate: 2% per epoch (Poisson).

10.2 Results

The Gini coefficient of 0.31 compares favorably to Bitcoin mining (Gini ≈ 0.87 [12]). The difference is attributable to the sublinear stake boost ($\log_2$) and the quality-weighted reward that prevents hardware dominance alone from determining earnings.

10.3 Sensitivity Analysis

11 Related Work

Compute token protocols. Render Network [6] rewards GPU providers for rendering but lacks training verification. Akash [7] provides a compute marketplace without quality scoring. Bittensor [5] uses inference scoring but does not coordinate training.

Metric	Value
Mean reward per node per epoch	8.4 GYM
Median reward	7.1 GYM
Gini coefficient (epoch 10,000)	0.31
Total slashing events	142 (1.42%)
Sybil attempts detected	7
Deflationary crossover (epoch)	4,800 ($\approx$ year 5)

Table 8: Simulation results over 10,000 epochs.

Parameter	Low	Default	High
Quality exponent α	1.0	1.5	2.0
Gini at α	0.38	0.31	0.27
Burn rate	5%	10%	20%
Deflation crossover (yr)	7.2	5.0	3.1
Spot-check rate	5%	10%	20%
Poisoning detected (%)	41%	65%	88%

Table 9: Sensitivity analysis of key parameters.

Proof-of-useful-work. Primecoin [8] mines prime chains; Gridcoin [9] credits BOINC compute. Neither addresses ML training specifically.

Token bonding curves. Bancor [10] introduced automated market makers; our emission schedule is simpler (halving) but achieves similar deflationary dynamics.

Staking economics. Ethereum 2.0 [11] uses quadratic leak for inactivity. Our progressive slashing is linear in offense count but quadratic in slash percentage, which is more aggressive against repeat offenders.

12 Conclusion

The tokenomics of Zoo Gym align compute provider incentives with model quality through three mechanisms: (1) a reward function that superlinearly weights delta quality, making honest participation the dominant strategy; (2) staking with sublinear boost, preventing Sybil and stake concentration attacks; (3) progressive slashing with bounty rewards, creating a quality auditing market. Simulation shows that the system reaches deflationary equilibrium by year 5 with moderate reward inequality (Gini 0.31), distributing value broadly across heterogeneous GPU providers.

Acknowledgments

This work was supported by Zoo Labs Foundation.

References

- [1] T. B. Brown, B. Mann, N. Ryder, et al., “Language Models are Few-Shot Learners,” *NeurIPS*, 2020.
- [2] Z. Shao, P. Wang, Q. Zhu, et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv:2402.03300*, 2024.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [4] N. Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” *ICLR*, 2017.
- [5] Opentensor Foundation, “Bittensor: A Peer-to-Peer Intelligence Market,” Whitepaper, 2023.
- [6] Render Network, “Decentralized GPU Rendering,” Whitepaper, 2020.
- [7] Akash Network, “Decentralized Cloud Compute Marketplace,” Whitepaper, 2020.
- [8] S. King, “Primecoin: Cryptocurrency with Prime Number Proof-of-Work,” 2013.
- [9] Gridcoin, “Rewarding BOINC Contributions with Blockchain,” Whitepaper, 2014.
- [10] E. Hertzog, G. Benartzi, and G. Benartzi, “Bancor Protocol: Continuous Liquidity for Cryptographic Tokens,” 2017.
- [11] V. Buterin et al., “Ethereum 2.0 Beacon Chain Specification,” 2020.
- [12] B. S. Srinivasan, “Quantifying Decentralization,” 2017.
- [13] H. B. McMahan et al., “Communication-Efficient Learning of Deep Networks from Decentralized Data,” *AISTATS*, 2017.
- [14] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-Robust Distributed Learning,” *ICML*, 2018.
- [15] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.
- [16] H. Touvron et al., “LLaMA 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv:2307.09288*, 2023.
- [17] T. Detrmers et al., “QLoRA: Efficient Finetuning of Quantized Language Models,” *NeurIPS*, 2023.
- [18] J. Benet, “IPFS – Content Addressed, Versioned, P2P File System,” *arXiv:1407.3561*, 2014.
- [19] Protocol Labs, “Filecoin: A Decentralized Storage Network,” Whitepaper, 2017.
- [20] L. von Werra et al., “TRL: Transformer Reinforcement Learning,” <https://github.com/huggingface/trl>, 2022.
- [21] V. Buterin, “Ethereum: A Next-Generation Smart Contract Platform,” Whitepaper, 2014.
- [22] A. Sergeev and M. Del Balso, “Horovod: Fast and Easy Distributed Deep Learning in Tensor-Flow,” *arXiv:1802.05799*, 2018.

- [23] P. Patarasuk and X. Yuan, “Bandwidth Optimal All-Reduce Algorithms for Clusters of Workstations,” *J. Parallel Distrib. Comput.*, 2009.
- [24] Unsloth, “Unsloth: 2x Faster LLM Fine-Tuning,” <https://github.com/unslothai/unsloth>, 2024.
- [25] D. J. Bernstein et al., “High-Speed High-Security Signatures,” *J. Cryptographic Engineering*, 2012.
- [26] J. Bai et al., “Qwen2.5 Technical Report,” *arXiv:2412.15115*, 2024.
- [27] R. Rafailov et al., “Direct Preference Optimization,” *NeurIPS*, 2023.
- [28] T. Dao, “FlashAttention-2,” *arXiv:2307.08691*, 2023.
- [29] Gensyn, “Gensyn Protocol: Trustless Training of Deep Neural Networks,” Whitepaper, 2023.
- [30] P. Blanchard et al., “Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent,” *NeurIPS*, 2017.